

1. Introduction

BlogNest (internally developed as the ContentGen backend) is a secure and intelligent REST API platform built to simplify blog creation, content management, and AI-assisted writing. The system is built using Node.js and Express.js, with MongoDB as the primary database managed through the Mongoose ODM for schema validation and efficient data modelling.

The platform integrates Google Gemini AI to power features such as AI-based blog content generation, AI FAQ assistance, and automated content summarization, allowing users to produce high-quality, context-aware content with minimal manual effort. Security is implemented through JWT-based stateless authentication, role-based access control (Admin, Editor, Author, Reader), and bcrypt.js password hashing, along with input sanitisation, rate limiting, and structured logging to guard against common web vulnerabilities.

2. Case Study

2.1 Background

Priya is a content creator who manages multiple blogs but struggles to organise posts, schedule publications, manage categories and tags, and track reader engagement efficiently.

2.2 Problem Statement

- Difficulty managing multiple drafts, categories, and tags.
- Limited visibility into post performance and engagement analytics.
- Time-consuming content approval and comment moderation processes.
- No streamlined system for handling media uploads or metadata.
- Lack of collaborative permission tiers for editors, authors, and administrators.

2.3 Solution

BlogNest provides a centralised, secure API layer that decouples business logic from the frontend. It introduces an internal permission matrix that governs content scheduling, category taxonomy, media path registration, and granular user roles.

2.4 Usage

- Author: Creates blog posts, uploads media, organises drafts, and reviews post-performance data.
- Editor/Admin: Reviews content queues, changes publishing status, moderates comments, and manages user roles.
- Reader: Views published posts and submits comments.

2.5 Outcome

By offloading workflow control to a robust backend, content execution delays are minimised, data integrity is enforced through schema validation, and role-based security guarantees that only authorised users perform administrative actions.

3. Requirements

3.1 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js (v16 or above) and npm (v8 or above).
- Express.js for backend routing.
- MongoDB for document-based data storage.
- Postman for API testing and validation.

- Visual Studio Code (or similar IDE).

3.2 Hardware Requirements

- Processor: Intel Core i5 (8th Gen) / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum (16 GB recommended).
- Storage: 1 GB of available disk space.

4. System Architecture

BlogNest follows a layered server architecture in which incoming requests pass through filtering and middleware before reaching the data services.

4.1 Architecture Components

- Interface: API clients such as Postman send requests to specific route paths.
- Express Server Gateway: Handles HTTP methods, parses JSON payloads, and manages CORS.
- Authentication & Role Matrix Middleware: Validates JWT tokens and checks role-based permissions.
- Service Layer: Enforces business rules for blog lifecycles, comment queues, and reporting.
- Database Interface Layer: Mongoose translates logic into MongoDB queries.
- AI Flow Service: Connects to Google Gemini AI via endpoints such as /api/ai/generate-blog and /api/ai/summarize for content generation and summarisation.

5. Data Model

5.1 User Entity

- `_id` — Primary Key (ObjectId)
- `name` — String, Required
- `email` — String, Required, Unique
- `password` — String, Hashed, Required
- `role` — String, Default: "reader"

5.2 Post Entity

- `_id` — Primary Key (ObjectId)
- `userID` — Foreign Key referencing User
- `name` — String, Required (Title)
- `photo` — Media pathway reference
- `message` — String (content body)
- `likes` — Number, Default: 0

5.3 Relationships

- User → Post: One-to-Many (an author can create multiple posts).
- User → Comment: One-to-Many (a user can post multiple comments).
- Post → Comment: One-to-Many (a post can have multiple comments).

6. Key Features

- Role-Based Route Guarding: Restricts unauthorised users from modifying protected settings.
- CRUD Blog Engine: Full lifecycle management of posts, statuses, and media.
- Categorisation System: Tag-based filtering for improved content discovery.

- **Comment Moderation:** Protected endpoints to manage approved, pending, and spam comments.
- **Sanitisation Safeguards:** Protection against XSS and NoSQL injection attacks.
- **AI-Driven Blog Generation:** Generates full blog posts from high-level topic prompts using an LLM.
- **Intelligent Summarisation:** Automatically generates concise summaries of long articles.

7. Roles and Responsibilities

- **Admin:** Full system control — manages users, roles, comment moderation, and logs.
- **Editor:** Reviews pending posts, publishes content, and manages categories.
- **Author:** Creates and manages posts belonging to their own account.
- **Reader:** Views published content and posts comments.

8. MVC Architecture

The system follows the Model-View-Controller pattern to decouple components:

- **Model Layer:** Mongoose schemas define data structure, validation, and defaults for MongoDB collections.
- **Controller Layer:** Processes incoming requests, applies business logic, and returns JSON responses.
- **View Layer:** As a headless API, this layer is represented by the routing network that maps endpoints to controller logic.

9. Implementation Overview

9.1 Project Setup

- Create a project folder with a Server subfolder and open it in VS Code.
- Run `npm init -y` inside the Server folder.
- Create `server.js` and the `models`, `controllers`, and `routes` directories.

9.2 Database Configuration

- Install Mongoose (`npm install mongoose`).
- Establish the MongoDB connection in a dedicated config file.
- Define Mongoose schemas and models for User and Post entities.

9.3 Testing

The backend was tested using Postman/ThunderClient after starting the server with `npm start`. Endpoints tested include user registration, user login, blog creation, blog retrieval, AI blog generation, and blog summarisation, confirming correct request validation and response handling across all role-protected routes.

10. Technology Stack Summary

The table below summarises the core technologies used across the BlogNest platform and the purpose each one serves within the overall architecture.

Technology	Purpose
Node.js	JavaScript runtime that executes the server-side application logic.
Express.js	Lightweight web framework used to define REST API routes and middleware.
MongoDB	NoSQL document database that stores users, posts, categories, and comments.

Mongoose ODM	Provides schema definition, validation, and query abstraction for MongoDB.
Google Gemini AI	Powers AI blog generation, FAQ assistance, and content summarisation.
JWT	Issues and verifies stateless authentication tokens for protected routes.
bcrypt.js	Hashes and verifies user passwords securely before storage.
Postman	Used to test and validate API endpoints during development.

11. Future Enhancements

While the current implementation of BlogNest covers the core requirements of an AI-assisted content management backend, several enhancements can further extend its capabilities and scalability:

- **Analytics Dashboard:** Introduce detailed engagement analytics (views, shares, read time) with visual charts for authors and admins.
- **Scheduled Publishing:** Allow authors to schedule posts for automatic publication at a future date and time.
- **Multi-language AI Generation:** Extend the AI blog generation pipeline to support content creation in multiple languages.
- **Image Generation Integration:** Combine text generation with AI image generation to auto-create featured post images.
- **Caching Layer:** Introduce Redis caching for frequently accessed public post listings to improve response times.
- **Notification System:** Real-time notifications for comment replies, post approvals, and role changes using WebSockets.
- **Unit and Integration Testing:** Expand automated test coverage using Jest and Supertest for higher reliability.
- **Containerisation:** Package the application using Docker for consistent deployment across environments.

12. Conclusion

BlogNest successfully delivers a secure, scalable, and AI-enhanced backend for blog content management. By combining a layered Express/MongoDB architecture with JWT-based role authorisation and Google Gemini AI integration, the system streamlines content creation, moderation, and publication workflows while maintaining strong data integrity and security. The project demonstrates a practical application of the MVC pattern and modern REST API design principles to solve real-world content management challenges.